

HELEP — L4 Design Process Document

Software Architecture — Orchestration with Kubernetes Mini Project

1. Project Specification

HELEP (Help Emergency Location and Protection) is a cloud-native emergency response platform designed for urban Cameroon. Citizens facing danger can trigger an SOS from their mobile device, sharing their GPS location and optionally capturing audio/video evidence. The system automatically identifies and dispatches the nearest available responder, notifies relevant parties, and generates real-time analytics for police and administrators. Primary users are: **citizens** (SOS senders), **responders** (police/volunteers who receive assignments), **police administrators** (who monitor zone heatmaps and incident statistics), and **system admins** (who manage accounts and credibility scores). The core business value is reducing emergency response time through automated, intelligent dispatch while preserving an audit trail of all events for accountability and analytics.

2. Requirements Analysis

2.1 Functional Requirements

#	Requirement	Service
F1	Citizens can register, log in, and manage emergency contacts	user-service
F2	Citizens can trigger an SOS with GPS coordinates and media capture (simulated)	data-service
F3	Citizens can cancel an active SOS incident	sos-service
F4	System assigns nearest available responder to an SOS automatically	dispatch-service
F5	System enforces single-responder-per-incident (no double dispatch)	dispatch-service
F6	Responders receive assignment notifications (simulated SMS/push)	notification-service
F7	System detects when an incident occurs in a danger zone and alerts nearby responders	dispatch-service
F8	Police can view event stream statistics and zone heatmaps	analytics-service

2.2 Non-Functional Requirements (NFRs)

NFR	Quality Attribute	Measurable Acceptance Criterion
NFR1	Availability	99.5% uptime; pod crashes auto-recovered by K8s in <30s via liveness probes
NFR2	Reliability	99% of SOS notifications delivered within 1s under 100 req/s (at-least-once Kafka delivery)
NFR3	Scalability	Any service scales horizontally to 5 replicas without code change (stateless + K8s HPA)
NFR4	Confidentiality	All endpoints require JWT Bearer token; JWT_SECRET stored in K8s Secret, never in code
NFR5	Integrity	No duplicate responder assignments; enforced by atomic SQL UPDATE WHERE busy=0
NFR6	Usability	All API responses return JSON within 200ms for non-Kafka operations
NFR7	Compatibility	Services containerised and deployable on any K8s cluster (Minikube, EKS, GKE, AKS)

2.3 Constraints

Constraint	Architectural Risk
Single responder per incident at any moment	Race condition risk when multiple SOS events arrive concurrently — mitigated by atomic DB UPDATE
Trigger-to-notify latency < 1 second	Synchronous HTTP fan-out would be too slow — forces event-driven async architecture via Kafka
24-hour build budget	No time for a real Outbox relay, real SMS provider, or real media upload — all simulated with services
SQLite per service for dev	Must convert to PersistentVolumeClaim in K8s; SQLite is not horizontally scalable beyond single node

3. Architectural Drivers & ASRs

From the NFRs and constraints, three Architecturally Significant Requirements (ASRs) are identified as the primary drivers — they have the highest impact on structural decisions.

ASR-1: Sub-second Trigger-to-Notify Latency (NFR2 + Constraint 2)

This is the dominant driver. It rules out synchronous HTTP chains (each hop adds latency) and mandates an event-driven architecture. Kafka with small JSON payloads and acks=all idempotent producer achieves sub-second delivery on a local broker. Partition keying (key=incident_id) preserves ordering without sacrificing throughput.

ASR-2: No Double Dispatch (NFR5 + Constraint 1)

The business rule that exactly one responder handles one incident at a time requires atomicity guarantees. The atomic UPDATE responders SET busy=1 WHERE busy=0 in dispatch-service/app/db.py enforces this at the DB level. Kafka partition ordering ensures only one consumer processes sos.triggered for a given incident_id at a time, preventing race conditions across replicas.

ASR-3: Independent Scalability of Services (NFR3 + NFR1)

Different services have different load profiles: sos-service spikes during emergencies, analytics-service is read-heavy. Microservices + K8s Deployments allow per-service replica scaling (HPA). Stateless services (JWT auth, SQLite on PVC) ensure any pod replica can handle any request without session affinity.

4. Component Identification

4.1 SRS-listed Components

The SRS identifies 8 logical components: User Management, Emergency Component, Incident Report & Response, Localization, Alert Management, Alert Delivery, Feedback & Review, Analytics & Statistics.

4.2 Service Decomposition (5 Services)

The 8 SRS components collapse into 5 minimal-viable services for the 24-hour budget:

Service	Port	SRS Components Merged	Justification
user-service	8001	User Management	Identity + JWT + credibility are cohesive; no reason to split
sos-service	8002	Emergency Component (triggered SOS)	SOS lifecycle is a single bounded context; split from dispatch to allow independent scaling
dispatch-service	8003	Incident Report & Response + Localization + Alert Management (zone alerting)	Dispatch is tightly coupled to assignment and localization
notification-service	8004	Alert Delivery	Delivery channel (SMS/push) is an isolated concern; decoupled via Kafka
analytics-service	8005	Analytics & Statistics	Read model separated from write path (CQRS); police queries should not block SOS

Feedback & Review is out of scope for the 24-hour build; marked as a bonus extension.

5. Architectural Style — Choice & Justification

The prescribed style is **Microservices + Event-Driven Architecture (EDA)** via Apache Kafka. Below we defend this choice against two alternatives.

Alternative 1: Monolithic Architecture

Criterion	Monolith	Microservices + EDA (chosen)
ASR-1 (sub-second latency)	Could achieve with in-process calls but tight coupling makes plastic failure; leader to isolate	Kafka poll, not a block
ASR-2 (no double dispatch)	Atomic DB write is easier in one DB	Achieved via partition keying + atomic SQL; slightly more complex but
ASR-3 (independent scaling)	Cannot scale individual components — entire monolith scales as one	Each service scales independently via K8s Deployment replicas
Dominant trade-off	Simpler to build, harder to scale and maintain	More operational complexity (Kafka, K8s) but satisfies all ASRs including

Alternative 2: Service-Oriented Architecture (SOA) with Synchronous HTTP

SOA with REST/gRPC between services would satisfy F1-F8 functionally, but fails ASR-1: a synchronous chain sos-service → dispatch-service → notification-service introduces cumulative HTTP latency (~50-200ms per hop) making sub-second delivery unlikely under load. It also creates tight temporal coupling — if dispatch-service is slow, sos-service blocks. EDA via Kafka eliminates temporal coupling: sos-service publishes and returns immediately (201); dispatch-service processes asynchronously. This is the dominant trade-off that drives the EDA choice.

6. Architectural Patterns Applied

Pattern	File:Line	Problem Solved in HELEP
Choreographed Saga	sos-service/app/main.py:88 dispatch-service/app/main.py:68	Coordinates multi-step emergency workflow without central coordinator; supports
Pub/Sub (Kafka)	every service/app/events.py:28,75	Decouples producers from consumers; fan-out to multiple services from single
Repository	every service/app/db.py:1-end	Isolates SQLite access; enables swap to PostgreSQL without touching business
Strategy	dispatch-service/app/matching.py:28-34	Pluggable responder selection (nearest, credibility, round-robin) switchable
Outbox-lite	sos-service/app/main.py:85-98	Reduces dual-write gap: DB insert then Kafka publish in same async block
Circuit Breaker	every service/app/events.py:55-72	Prevents cascade failure when Kafka broker is temporarily unreachable
CQRS	analytics-service/app/main.py,db.py	Separates read model (analytics) from write path; police queries do not cont
Bulkhead	each service/app/main.py GROUPS	Separate Kafka consumer groups isolate failure — slow analytics does not s

See patterns.pdf for full code citations and detailed explanations of each pattern.

7. Architecture Decision Records (ADRs)

ADR-001: Kafka Partition Count = 3, Keyed by incident_id

Context:

The saga requires that all events for a single SOS incident (sos.triggered, responder.assigned, safety.zone.entered, sos.cancelled) are processed in order. With multiple replicas of dispatch-service, two replicas could process the same incident concurrently, breaking the no-double-dispatch invariant.

Decision:

Set KAFKA_NUM_PARTITIONS=3 and key every saga publish with the incident_id (e.g. publish('sos.triggered', payload, key=iid)). Kafka guarantees all messages with the same key land on the same partition. Within a consumer group, one partition is owned by one consumer at a time — so all events for one incident are processed sequentially by one pod.

Consequences:

Ordering is preserved per incident; no distributed locks needed. Maximum parallelism is bounded by partition count (3). Three partitions support up to 3 concurrent incidents being dispatched in parallel, which is sufficient for the demo but would need increasing in production.

Alternatives Considered:

1 partition: simpler but serialises all incidents globally — kills throughput. 10 partitions: more parallelism but wastes resources on a single-broker dev setup.

ADR-002: SQLite per Service vs. Shared PostgreSQL

Context:

Each service needs persistent state: users, incidents, assignments, notifications, analytics counters. Options are a shared PostgreSQL instance or a dedicated database per service.

Decision:

Use SQLite per service with a PersistentVolumeClaim in K8s. Each service's DB_PATH is mounted from its own PVC (user-data, sos-data, dispatch-data, etc.). Services are completely isolated at the storage level — no service can read another's tables.

Consequences:

Service isolation is enforced at the infrastructure level. Schema changes in one service cannot break others. The trade-off is that SQLite is single-writer — horizontal scaling of any service beyond one replica requires migrating to PostgreSQL or an embedded distributed DB. For the 24-hour demo scope this is acceptable.

Alternatives Considered:

Shared PostgreSQL: simplifies cross-service queries but violates bounded context isolation and creates a central point of failure. All services would depend on one DB's availability.

ADR-003: Plain K8s Deployments vs. Helm Umbrella Chart

Context:

Five services + Strimzi Kafka need to be deployed to Kubernetes. Options are raw YAML manifests per service or a Helm umbrella chart that templates all services.

Decision:

Use plain Kubernetes YAML manifests (one file per service in k8s/services/, Kafka in k8s/kafka/). Each manifest contains a PVC + Deployment + Service. Environment-specific values (KAFKA_BOOTSTRAP, JWT_SECRET) are injected via kubectl set env and K8s Secrets rather than Helm values.

Consequences:

Plain YAML is immediately readable and requires no tooling beyond kubectl. The trade-off is duplication — the five service manifests are structurally identical except for name, port, and image. A Helm chart with a loop over a services list would eliminate this duplication and enable environment-specific values.yaml overrides (dev/staging/prod). For a single-environment demo, plain YAML is sufficient.

Alternatives Considered:

Helm umbrella chart: better for multi-environment production deployments. Kustomize overlays: a middle ground — no templating engine but supports patch overlays per environment.

8. Trade-offs & Improvement Perspectives

Weakness 1: SQLite Single-Writer Bottleneck

SQLite is a single-writer database. If dispatch-service is scaled to 3 replicas (K8s HPA), all three pods share one PVC mount and SQLite will serialize writes, eliminating horizontal scaling benefits and risking file lock contention. **Fix:** Migrate each service's storage to a dedicated PostgreSQL instance (one per service, deployed as a K8s StatefulSet with its own PVC). PostgreSQL supports concurrent writers and connection pooling via PgBouncer, enabling true horizontal scaling.

Weakness 2: Outbox-lite Dual-Write Gap

The current Outbox-lite pattern (db write then Kafka publish in the same async block) has a small window where the DB write succeeds but the process crashes before Kafka publish. The incident row exists but no sos.triggered event is ever emitted — the SOS is silently lost. **Fix:** Implement a true Transactional Outbox: write the event to an outbox table in the same SQLite transaction as the incident row. A separate relay goroutine polls the outbox table, publishes to Kafka, and marks the row as sent. This eliminates the dual-write gap entirely.

Weakness 3: No API Gateway / Ingress for External Access

Currently each service is exposed only via kubectl port-forward — there is no single entry point for external clients. A mobile app cannot reach the services without knowing internal cluster IPs. **Fix:** Deploy an NGINX Ingress Controller (minikube addons enable ingress) with path-based routing: /api/users → user-service:8001, /api/sos → sos-service:8002, etc. This implements the API Gateway pattern, provides a single TLS termination point, and enables rate limiting and authentication middleware at the edge.

All architectural choices are traced to SRS requirements, NFRs, or ASRs as required. Source code: github.com/MATRIX30/Helep | K8s deployment: Minikube v1.38.1, Kubernetes v1.35.1, Strimzi Kafka 4.1.0